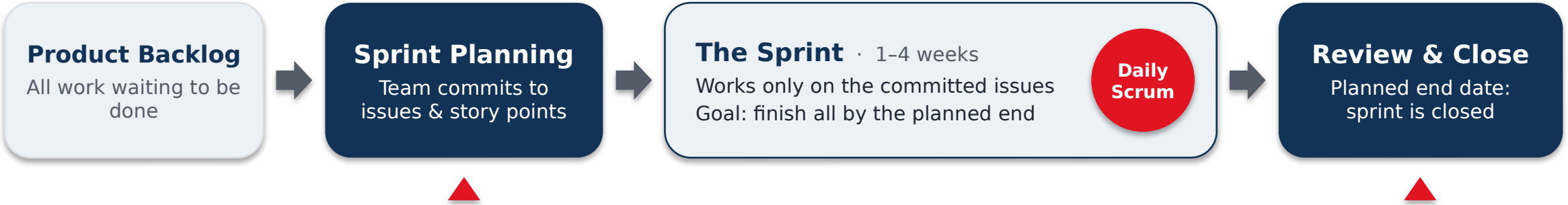
Dataset & Research Story

Predicting Late Sprint Closure and Unfinished Committed Scope from Reconstructed Jira Histories

HOW A SCRUM SPRINT WORKS

Scrum teams work in short, fixed time-boxes called sprints.



WE PREDICT HERE
using only what was known at sprint start

1 Late closure?
Sprint closes >1 day after its planned end

2 Spill over?
A committed issue is still unresolved at the end

Can we tell from the planning data which sprints will struggle?

TAWOS is a public dataset of Agile open-source projects, taken from Jira. We picked eight of them and try to predict, at sprint planning, which sprints will close late or leave work unfinished.

The hard part: working out what the team knew on day one, without peeking at what happened later.

2,657
original sprint records

145,309
issues

8
open-source projects

Core tables Sprint | Issue | Change_Log

Maryam Hanif • Hani AbuSharkh • Samra Nawazish • Najwa Suleiman • Alyah Alameeri

GitHub: github.com/hani-ziad/SwEng_202610

Data: TAWOS, doi.org/10.5522/04/21308124 (Apache-2.0)

Handling the Data

First rebuild what was known at planning time. Then model only that.

2,657 Raw sprints → **2,610** Eligible closed → **− 8** Cold starts → **2,602** Modeled sprints

20,603 reconstructed issue–sprint commitments

1 FILTER

Drop FUTURE and ACTIVE sprints and sprints with impossible timestamps. Drop one true cold start per project.

2 RECONSTRUCT

Replay the Jira change history back to the sprint start to see which issues were in the sprint and their story points.

3 PREVENT LEAKAGE

Only use what was known at sprint start.
We split by time inside each project: earlier 70% to train, later 30% to test.

Main data problem

Some issues have no recorded Sprint or Story Points changes. We use the current Jira value for those, and we re-ran the models to check that this does not drive the results.

Test sets: 786 sprints (late closure), 511 (spillover, nonempty). Class balance: about 31% late and 94% spillover among nonempty sprints, so ROC-AUC is the headline metric and F1 is shown for reference.

Algorithms & Experimental Setup

A simple baseline, simple models and tree models, all tested the same way on a time split

ALWAYS POSITIVE / PREVALENCE

Baseline

Reference point: what you get with no learning at all

SCOPE-ONLY

Logistic Regression

Issues + Story Points + sprint length

Simple and readable: does the starting scope alone carry most of the signal?

transparent baseline

FULL FEATURE MODEL

Random Forest

Scope + recent history + project identity

Flexible, non-linear: do history and project add anything on top of scope?

300 trees • depth 5

FULL FEATURE MODEL

XGBoost

Same full predictor set

Another tree model: does boosting beat the random forest?

300 trees • depth 4 • lr .05

Evaluation

ROC-AUC
Primary ranking metric

0.5
Fixed threshold

70 / 30
Earlier vs later sprints, per project

Same pipeline
Fair preprocessing

Also reported: F1 • Precision • Recall • PR-AUC • Brier score

Results

The two outcomes behave differently: late closure needs context, spillover is mostly about scope

Outcome	Model	F1	ROC-AUC	Train time	Model size
Late closure	Always-positive	.477	.500	0 s	no fitted model
Late closure	Scope-only LR	.336	.530	0.0053 s	4 coefficients
Late closure	Logistic Regression	.453	.636	0.0081 s	16 coefficients
Late closure	**Random Forest**	**.562**	**.705**	**0.4389 s**	**300 trees / 15,374 nodes**
Late closure	XGBoost †	.535	.695	1.7950 s †	300 trees, depth 4
Spillover	Always-positive	.971	.500	0 s	no fitted model
Spillover	**Scope-only LR**	**.696**	**.885**	**0.0061 s**	**4 coefficients**
Spillover	Logistic Regression	.763	.829	0.0122 s	16 coefficients
Spillover	Random Forest	.875	.852	0.3557 s	300 trees / 10,308 nodes
Spillover	XGBoost †	.963	.862	— †	300 trees, depth 4

LATE CLOSURE

0.705 AUC

Random Forest ranks best here. XGBoost is close behind at 0.695.

SPILLOVER

0.885 AUC

The simple scope-only model ranks best, ahead of the Random Forest at 0.852.

Late closure uses all sprints (test n = 786). Spillover uses sprints with committed scope (test n = 511). Every model gets the same split, preprocessing and seed (42). Train time is the median of 5 fits. † We could not run XGBoost where we built this table, so its scores are from our saved run. The 1.795 s was timed by the team with xgboost installed.

Why? Explanation + Ablation

The best result comes from a property of the data, not from a fancier model

Late closure: RF wins (AUC .705)

A weaker signal that depends on the project. Late sprints start bigger (on time → late):

7.3 → 8.8
mean issues

32.9 → 38.3
mean story points

Project identity + sprint length contribute most.

Spillover: Scope-only LR wins (AUC .885)

How much work was committed is what matters.

-0.220 AUC
when committed issue count is shuffled

Next-largest drop: only 0.021

Feature-group ablation

Random Forest features	Spillover AUC (n = 511)	Late closure AUC (n = 786)
Full	.852	.705
Planning only	.871	.706
History only	.636	.625
No project identity	.866	.696
No committed scope	**.652**	**.690**
Scope-only LR (reference)	**.885**	**.530**

What the ablation shows

Our guess: if starting scope drives spillover, taking it out should break the ranking. It does. RF AUC falls from .852 to .652 without scope, well below the 3-feature scope-only LR (.885). For late closure the scope-only model is at chance (.530) and dropping scope barely matters (.705 → .690).

Takeaway: how much work a sprint starts with explains most of the unfinished-work signal (shuffling committed issue count costs 0.220 AUC). Late closure stays weaker and depends on the project (shuffling project costs 0.104).